

Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Honours (BPhil) 2026

The University of Western Australia

Supervisors: Dr Michael Bertolacci & A/Prof Edward Cripps

Overview

Neural Bayes estimators learn the map from a dataset straight to its posterior estimate — giving fast, *amortised* Bayesian inference.

- **The catch:** training one is *itself* a Monte Carlo problem — so the gradient that drives training is **noisy**.
- **This thesis:** treat that as **variance reduction** — cut the noise, and train better.

Idea 1 · Rao–Blackwellisation

Replace noisy targets by their exact conditional mean. **Provably** lower variance — needs a *conjugate* block.

Idea 2 · Importance sampling

Draw from a proposal and reweight.
General — but with *no guarantee*.

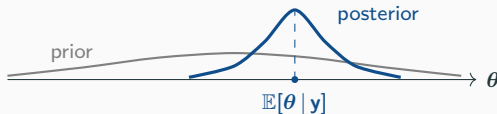
Background: Neural Bayes estimators

Bayesian inference

- Fix a model: parameters θ with prior $p(\theta)$, and data $\mathbf{y}$ with likelihood $p(\mathbf{y} | \theta)$.
- Seeing $\mathbf{y}$, Bayes' rule updates the prior into the **posterior distribution**:

$$p(\theta | \mathbf{y}) \propto p(\mathbf{y} | \theta) p(\theta).$$

- The posterior is our full, updated belief about θ .



Point estimation

- Often we do not want the whole posterior — just a single **point estimate** $\hat{\theta}$ of θ .
- The usual choice is the **posterior mean** $\hat{\theta} = \mathbb{E}[\theta | \mathbf{y}]$.
- But it is **rarely available in closed form** — so we need a **procedure** that turns $\mathbf{y}$ into $\hat{\theta}$:



The Bayes risk

Which point estimate $\hat{\theta}(\mathbf{y})$ should we return? Fix a **loss** $\ell(\theta, \hat{\theta})$ and minimise the **Bayes risk**

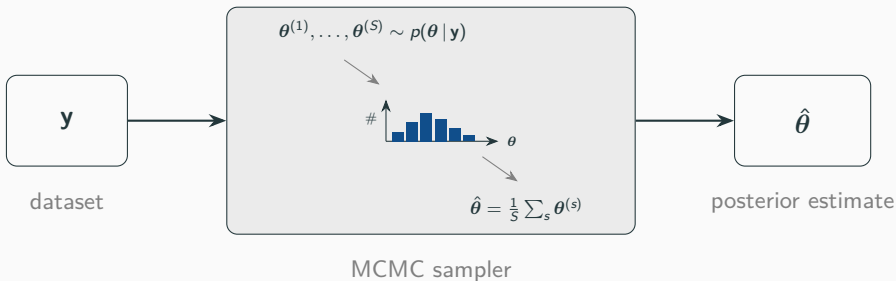
$$\mathbb{E}_{\theta, \mathbf{y}}[\ell(\theta, \hat{\theta}(\mathbf{y}))].$$

The choice of loss sets *which* posterior summary is optimal:

loss $\ell(\theta, \hat{\theta})$	optimal estimate (its minimiser)
squared $\ \theta - \hat{\theta}\ ^2$	posterior mean $\mathbb{E}[\theta \mathbf{y}]$
absolute $ \theta - \hat{\theta} $	posterior median
quantile $(\alpha - \mathbf{1}_{\{\theta < \hat{\theta}\}})(\theta - \hat{\theta})$	posterior quantile

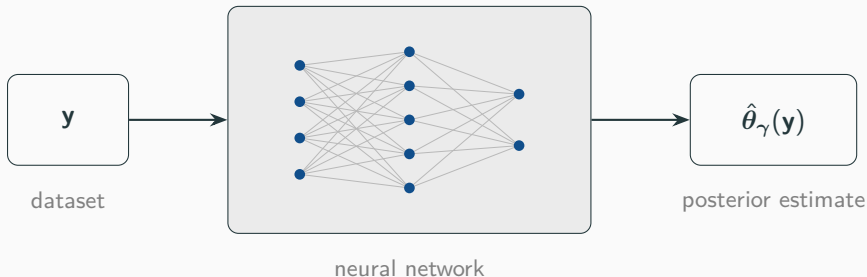
Usually this criterion is used to *choose* which estimator to use

Method (1): Markov chain Monte Carlo



- $\theta^{(1)}, \dots, \theta^{(S)}$ come from a **Markov chain** with the posterior as its stationary distribution — accurate and general, but can be **slow** to converge.
- And it must be **re-run from scratch for every new dataset y** — costly when you have many.

Method (2): Neural Bayes estimator



- A **neural network** $\hat{\theta}_\gamma(\cdot)$, trained *once* — a *neural Bayes estimator*.
- Training data is free: **simulate** pairs (θ, y) and adjust γ so $\hat{\theta}_\gamma(y) \approx \theta$.
- Then each new dataset is a single **forward pass** — the cost is **amortised**.

The idea: training is Monte Carlo

Back to the Bayes risk. **Goal:** choose the network weights γ that minimise

$$L(\gamma) = \mathbb{E}_{\theta, \mathbf{y}}[\ell(\theta, \hat{\theta}_{\gamma}(\mathbf{y}))].$$

No closed form — so each step estimates it by **Monte Carlo** on a minibatch and takes a **stochastic-gradient** step:

$$\hat{L}(\gamma) = \frac{1}{B} \sum_{b=1}^B \ell(\theta^{[b]}, \hat{\theta}_{\gamma}(\mathbf{y}^{[b]})), \quad \gamma \leftarrow \gamma - \eta \nabla_{\gamma} \hat{L}(\gamma).$$

(each minibatch draws fresh $(\theta^{[b]}, \mathbf{y}^{[b]}) \sim \text{model.}$)

- **Insight:** the minibatch gradient $\nabla_{\gamma} \hat{L}$ is a **random estimate** — it has variance, from *how we sample the loss*, not from the inference problem.
- So we can **apply variance-reduction techniques** to speed up and stabilise training.

Idea 1: Rao–Blackwellisation

The trick

The training target θ is a **noisy draw**. Split $\theta = (\theta_1, \theta_2)$; in the minibatch loss estimator, replace each term by its **conditional mean** given $(\theta_2^{[b]}, \mathbf{y}^{[b]})$ — averaging out the noise in θ_1 :

$$\hat{L}_{\text{MC}}(\gamma) = \frac{1}{B} \sum_{b=1}^B \ell(\theta^{[b]}, \hat{\theta}_{\gamma}(\mathbf{y}^{[b]})),$$

$$\hat{L}_{\text{RB}}(\gamma) = \frac{1}{B} \sum_{b=1}^B \mathbb{E}[\ell(\theta^{[b]}, \hat{\theta}_{\gamma}(\mathbf{y}^{[b]})) \mid \theta_2^{[b]}, \mathbf{y}^{[b]}].$$

- Same mean — but the **Rao–Blackwell theorem** makes $\hat{L}_{\text{RB}}$ have **never larger variance**, for any model and any architecture.
- Exactly computable when the block θ_1 is **conditionally conjugate**.

Squared error: a clean target

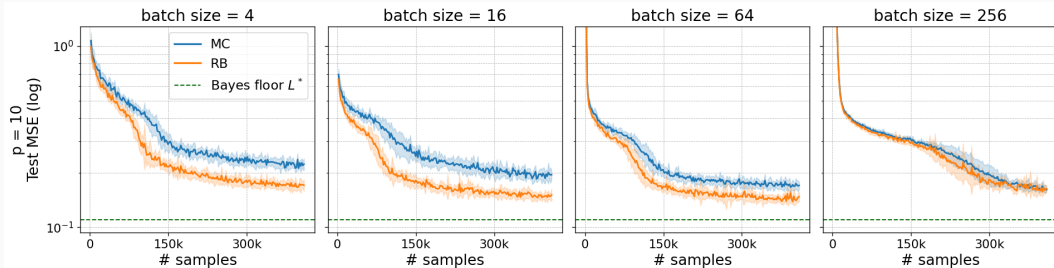
For **squared-error** loss the conditional mean is simple — expand each term of $\hat{L}_{\text{RB}}$:

$$\hat{L}_{\text{RB}}(\gamma) = \frac{1}{B} \sum_{b=1}^B \left[\|\hat{\theta}_{\gamma}(\mathbf{y}^{[b]}) - \mathbb{E}[\boldsymbol{\theta} \mid \boldsymbol{\theta}_2^{[b]}, \mathbf{y}^{[b]}]\|^2 + \underbrace{\text{Tr}(\text{Var}(\boldsymbol{\theta} \mid \boldsymbol{\theta}_2^{[b]}, \mathbf{y}^{[b]}))}_{\text{constant in } \gamma} \right].$$

- Just squared error to a **clean target** $\mathbb{E}[\boldsymbol{\theta} \mid \boldsymbol{\theta}_2^{[b]}, \mathbf{y}^{[b]}]$ — the exact conditional mean, in place of the noisy draw $\boldsymbol{\theta}^{[b]}$.
- The variance term is **constant in** γ : it drops straight out of the gradient.

Bayesian linear regression: take $\boldsymbol{\theta} = (\boldsymbol{\beta}, \sigma^2)$ and condition on σ^2 ; then $\boldsymbol{\beta}$ is Gaussian, so the clean target $\mathbb{E}[\boldsymbol{\beta} \mid \sigma^2, \mathbf{y}]$ is **closed-form**.

Does it pay off?



Bayesian linear regression, $p = 10$ coefficients; mean ± 1 s.d. over repeated training runs.

- The Gibbs sampler gives the Bayes-optimal floor L^* (green).
- RB reaches a **lower test error** at the small and moderate batches — where the training gradient is noisiest.
- The advantage **shrinks as the batch grows**.

Idea 2: Importance sampling

Importance sampling: A more general lever

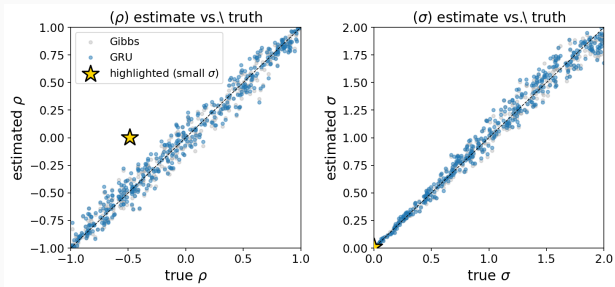
No conjugacy required — unlike Rao–Blackwell. Draw the parameters from a **proposal** $q(\theta)$ instead of the prior $p(\theta)$, then **reweight** to leave the risk unchanged:

$$L(\gamma) = \mathbb{E}_{\theta \sim p, \mathbf{y}}[\ell(\theta, \hat{\theta}_\gamma(\mathbf{y}))] = \mathbb{E}_{\theta \sim q, \mathbf{y}}[w(\theta) \ell(\theta, \hat{\theta}_\gamma(\mathbf{y}))], \quad w(\theta) = \frac{p(\theta)}{q(\theta)}.$$

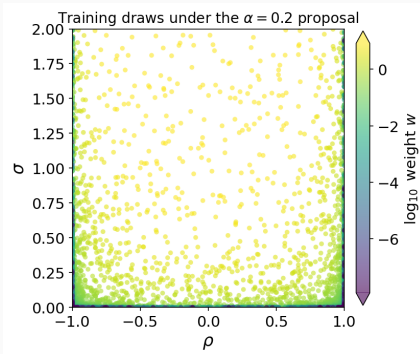
- Unbiased for **any** proposal q : the weight $w = p/q$ undoes the change of measure, so we still target the original prior.
- Point q at the **informative regions** — spend more of the budget where the estimator has the most to learn.
- But **no guarantee**: bad weights inflate variance and collapse the effective sample size.

A test case: AR(1)

AR(1) test: the extreme edges (tiny σ , $|\rho| \rightarrow 1$) *look* hardest, so we oversample them.

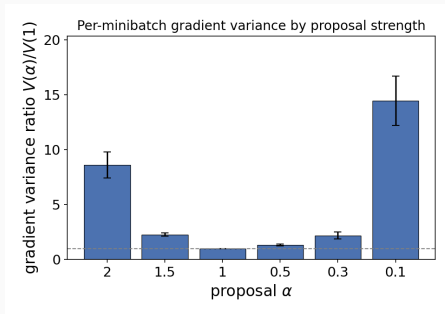


The GRU tracks Gibbs; the star (tiny σ) is **unidentifiable**.

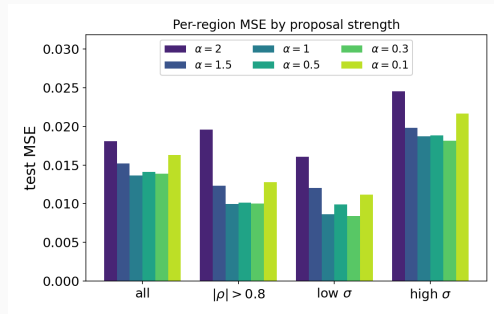


Draws pile on the edges.

Importance sampling: the result



Every proposal **inflates** the training-gradient variance ($V(\alpha)/V(1) \geq 1$) — the very quantity RB shrinks.



And the test error falls in **no** region — no proposal beats the uniform baseline ($\alpha = 1$).

Conclusion

Training a neural Bayes estimator involves Monte Carlo — so its training variance can be reduced.

- **Rao–Blackwellisation** (needs a conjugate block): a *provably* lower-variance training gradient, for any model or architecture — on linear regression it trains to a lower error, most where the batch is small.
- **Importance sampling** (needs nothing special): more general, but unguaranteed and problem-dependent — here it did not help.

Future work: many more variance-reduction techniques remain to be tried.

Thank you